Learning to Play Belote with Reinforcement Learning

Noa Curiel & Yoni Benabou

25 October 2026

Abstract

We trained an artificial intelligence to play two-player Belote without giving it any strategy: it learned everything by playing against itself, using the PPO method. It wins **83% of its games** against a player who plays at random and **62%** against the best strategy we wrote by hand. This result is reproduced across three independent training runs. We then added trump bidding: early results show **73.5% of games won** against that same strategy, and better bids than those of a hand-written rule.

1. Introduction

Goal

The project asks whether a program can learn to play Belote well **by itself**, only by playing, and rediscover or surpass the strategies of human players.

We use **reinforcement learning**. The program plays millions of games. After each one, it reinforces the choices that won it points and drops those that lost it points. Nobody explains strategy to it: it only knows the legal moves and the points won.

Why Belote

- **A widely played game.** It is one of the most popular card games in France, with a tournament held every year at the Technion.
- **A real challenge for an AI.** It is a game of **hidden information**: you cannot see your opponent's cards. In chess, everything is visible; here, every move is a bet on what the opponent holds, as in poker.
- **A large share of luck.** The deal weighs heavily on the outcome of a hand. Careful measurement methods are therefore needed to tell skill from luck.
- **A game we know.** As long-time players, we can judge whether the AI plays "intelligently".

What we built

We wrote everything ourselves in Python, about 4,400 lines: the game engine, the learning environment, four reference opponents, the PPO method, the measurement tools, 9 automated test suites and a web app to play against the AI.

2. The starting game: Coinche

Coinche is the variant of Belote that we play. It is played by four, in two teams of two sitting opposite each other, with a 32-card deck (Ace down to 7 in all four suits). Each player receives 8 cards: every card is dealt.

Bidding

Before play, players take turns announcing a **contract**: a number of points their team commits to scoring (80, 90, 100… up to 160, or "capot" for all the tricks), with the trump suit of their choice. Each bid must beat the previous one. Bidding ends when the other three players pass.

An opponent who thinks the contract will fail can **"coinche"**: the points at stake are doubled. The bidding team can then **"surcoinche"** to double them again.

Card play

A hand is played in 8 tricks. The first player leads a card, and each player plays in turn, following three obligations:

1. **Follow suit**: play a card of the suit led if you have one.
2. **Trump**: if you have none of the suit led, play a trump (unless your partner is already winning the trick).
3. **Overtrump**: when you play a trump, play one higher than those already on the table if you can.

The trick is won by the highest trump or, failing that, by the highest card of the suit led. The trick winner leads the next one.

Card values

The order and value of cards change in the trump suit: the Jack and the 9 rise above everything else.

Card	Points in trumps	Points in other suits
Jack	**20**	2
9	**14**	0
Ace	11	11
10	10	10
King	4	4
Queen	3	3
8	0	0
7	0	0

Trump order: Jack, 9, Ace, 10, King, Queen, 8, 7. Non-trump order: Ace, 10, King, Queen, Jack, 9, 8, 7.

Scoring

- The cards are worth 152 points in total, plus **10 points for the last trick** ("dix de der"), so 162 points per hand.
- The player holding **the King and Queen of trumps** scores 20 extra points ("belote, rebelote").
- If the bidding team reaches its contract, it scores its points plus the contract value. Otherwise the contract **fails**: the other team scores 160 points plus the contract value, doubled if coinched.
- The game is played to a target set in advance, often 1,000 or 2,000 points.

Scoring details vary from table to table; these are the most common rules.

3. Our two-player adaptation

Why adapt

We first programmed a complete four-player Coinche, which worked. But for learning, our professor advised us to simplify. With four players, the AI must guess three hidden hands, cooperate with a partner, share its reward with them and handle bidding. The link between a move and its outcome becomes too blurry, and learning too noisy.

So we adapted the game to two players, keeping what makes it hard: trumps, trumping and hidden information.

Coinche versus our version

	Coinche	Our two-player version
Players	4, in two teams	2, head to head
Cards in hand	8	8
Cards you cannot see	the hands of the other 3 players	the opponent's hand and **16 cards never dealt**
Choosing trumps	bidding with a contract, coinche	first drawn at random, then a simple bid on a turned-up card (section 11)
Bidder's commitment	reach the contract	score more points than the opponent
Obligations	follow suit, trump, overtrump	follow suit, trump; overtrump only when trumps are led
Card values	see above	identical
Dix de der, belote-rebelote	yes	yes
End of game	often 1,000 or 2,000 points	501 points, about 10 hands

With 16 cards never dealt, a card you have not seen is not necessarily in the opponent's hand: deductions are trickier than with four players, where every card is in play.

We first set trumps at random so the AI could learn card play. Bidding was added only once that first goal was reached.

4. The learning environment

For an AI to learn, the game must be framed as an "environment": at each turn, the AI receives a description of the situation, chooses an action, then receives a reward. We built this environment with Gymnasium, the standard library in the field.

What the AI sees

At each move, the AI receives a vector of **598 numbers** describing everything a human player can know:

Information	Size
The cards in its hand	32
The cards already played	32
The trump suit	4
Both players' scores	2
Who won each trick (me, the opponent, not yet played)	8
The detail of each trick: my card, the opponent's card, who led	520

The AI **never** sees the opponent's hand or the 16 undealt cards. Everything is described from its own point of view ("me" and "the opponent"): this is what lets the same program play either seat.

What it can do

It chooses one card out of 32. Cards forbidden by the rules are **masked** before it chooses: their probability is set to zero. It can therefore never play an illegal move, and does not have to learn the rules.

Its reward

It receives the points it actually wins: the points of each trick taken, the dix de der and belote-rebelote. These points are scaled between 0 and 1 to stabilise learning. The reward is therefore given hand by hand, not at the end of the 501-point game.

5. The reference opponents

To know whether the AI is improving, it must be compared with players of known strength. We wrote four players, from weakest to strongest. Each one keeps the previous one's strategy and adds one human-player idea.

Games: 1,000 duplicate games of 501 points (±2 percentage points). Hands: 10,000 duplicate hands (±0.7 percentage points). See section 7. **TrumpCount**, the strongest, is the main benchmark throughout the project: it is the opponent against which we compare every version of the AI.

Player	Strategy	Games won against Random (501 points)	Hands won against Random
Random	Plays a legal card at random. This is the floor.	51%	50%
Heuristic	Leads with its highest trump. Wins tricks with the lowest card possible. Otherwise, discards its lowest card.	62%	52.9%
VoidExploit	Spots the suits the opponent no longer holds and leads them to make it spend its trumps.	70.7%	56.0%
TrumpCount	Counts the remaining trumps and only leads trumps when it holds the majority.	**72%**	56.4%

6. The method: PPO and self-play

The neural network

The AI is a neural network, that is, a function with many internal settings that are adjusted little by little. It takes the 598 numbers describing the situation, passes them through two layers of 128 neurons, and gives two outputs:

- **the policy**: a probability of playing each legal card;
- **the value**: an estimate of the points it will still win in the hand.

At the start, its settings are random and it plays almost at random.

Self-play

The same network plays **both players**, alternating who leads. It therefore learns against an opponent that improves at exactly the same pace. It is shown no human games and no strategy.

PPO

PPO (*Proximal Policy Optimization*) is one of the most widely used reinforcement learning methods. We wrote it ourselves, piece by piece, rather than using an off-the-shelf library. Training repeats a four-step loop:



Figure 1: the training loop

1. **Play.** The AI plays 128 hands against itself and records every move.
2. **Reward.** Each move receives the points it earned.
3. **Judge.** For each move, the points obtained are compared with what the network predicted. The difference is called the **advantage**: positive if the move did better than expected, negative otherwise.
4. **Adjust.** Moves with a positive advantage become slightly more likely, the others slightly less. PPO **caps the size of each correction** (20% at most) so the network does not suddenly forget what it has already learned.

Every 50,000 games, training pauses to evaluate the AI against the four opponents and save the model.

Chosen settings

Setting	Value
Hands played before each update	128
Learning passes over those hands	4
Minibatch size (moves)	256
Learning rate	0.0003
Cap on each correction	20%
Duration of a 3-million-game training run	5 h 20 on the Technion machine

7. Measuring strength

In card games, luck blurs everything. Our best hand-written strategy beats random play in only 56% of hands. With a naive measurement over 200 games, the margin of error (± 7 percentage points) was larger than the improvements we wanted to detect. So we built three tools.

Duplicate deals

As in bridge tournaments, each deal is played **twice** with the hands swapped: the AI first plays hand A against hand B, then hand B against hand A. The luck of the deal cancels out, leaving only the difference in play.

Margins of error on the point difference per hand, measured over 4,000 hands; percentages over 10,000 duplicate hands. The margin of error is divided by 2 to 3: 4 to 9 times fewer hands are

Match-up	Hands won	Margin of error, independent deals	Margin of error, duplicate deals
Heuristic vs Random	52.9%	±1.44 points	±0.74 points
TrumpCount vs Random	56.4%	±1.40 points	±0.69 points
TrumpCount vs Heuristic	55.0%	±1.22 points	±0.52 points
VoidExploit vs Heuristic	54.6%	±1.20 points	±0.41 points

needed for the same precision. These percentages also show how hard the game is: even between two strategies of very different strength, the better one wins only 53 to 56% of hands.

The "cheating" player

When we played against the AI ourselves, we lost most games. Yet it won "only" 63% of hands against random play. To understand why, we programmed a player that **sees the opponent's hand** and computes the perfect move. No honest player can do better. Yet it wins only **76% of hands** against random play: within a single hand, a bad hand cannot be saved.

Full 501-point games

So we also measure strength as in a real game: first to 501 points, about 10 hands, in duplicate games. Over a whole game, luck evens out and skill shows clearly: the cheater wins 99% of games against random play.

We therefore give two figures for each match-up: the percentage of **hands** won and the percentage of **games** won. Game measurements use 1,000 games per match-up, accurate to ±2 percentage points.

8. Debugging and tuning

The first training runs ran without errors, but the AI barely improved. Two invisible bugs and a bad setting were the cause. Every fix is now checked by an automated test.

Two bugs that blocked learning

The mixed-up point of view. The winner of each trick was recorded as "player 0" or "player 1", while everything else was described as "me" and "the opponent". Since a single network plays both seats, it could not tell who had won the previous tricks. We found it by rereading the code, and re-encoded it as "me / opponent / not yet played".

Both players' points added together. To judge a move, the advantage calculation chained the moves of both players as if they belonged to one. In a game where one player's gain is the other's loss, the signal "this move won me points" cancelled out. We found it while looking for why the AI was not learning. The calculation is now done separately for each player.

Tuning: one hypothesis at a time

Once the bugs were fixed, the AI was still below TrumpCount. We formed four hypotheses and tested each over 200,000 games, changing only one thing at a time.

Configuration	Hands won vs TrumpCount	Difference vs TrumpCount	vs Random	vs Heuristic	vs VoidExploit
Start: win bonus, fast learning	47.9%	−4.6	+4.0	−0.8	−3.8
No win bonus	50.4%	−0.4	+9.8	+0.7	−2.0
No bonus + more exploration	51.7%	−0.6	+9.3	+1.5	−1.5
No bonus + training against the heuristics	51.0%	−0.1	+5.0	+8.3	+5.7
No bonus + slower learning	**52.9%**	**+1.0**	**+9.9**	**+2.6**	**+0.8**

Percentage of hands won against TrumpCount, then mean point difference per hand against each opponent, in duplicate deals, after 200,000 training games.

- **Removing the win bonus is the decisive change.** We added ±1 to the reward depending on whether the hand was won or lost. But winning a hand depends mostly on the deal: this bonus drowned the real signal in noise.
- **Slower learning adds to it well.** Corrections that were too abrupt made the network oscillate. It is the only configuration that is positive against all four opponents.
- **More exploration brings nothing** once the bonus is removed.
- **Training against the heuristics specialises the AI**: it crushes them but plays worse overall. Self-play remains the best choice.

The "no bonus + slower learning" configuration became our default setting for all later training runs.

9. Results without bidding

In real 501-point games, **the AI wins 83% of its games against random play and 62% against TrumpCount**, our best hand-written strategy. It beats all four reference opponents.

The main result: 501-point games

A 501-point game lasts about 10 hands: the luck of the deal evens out, and playing strength shows clearly (section 7).

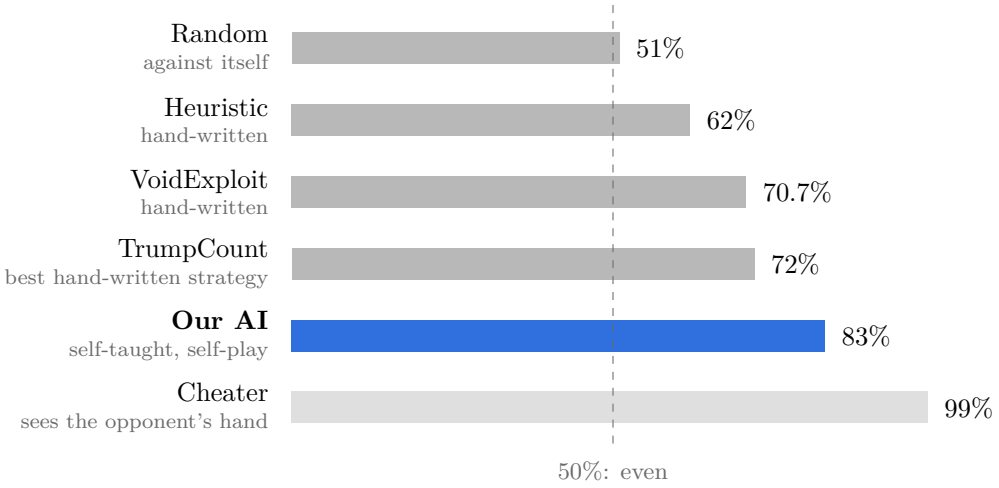


Figure 2: 1,000 duplicate games per player (400 for the cheater), ±2 percentage points

Against random play, the AI does better than all our hand-written strategies: 83% versus 72% for TrumpCount. It sits between those strategies and the theoretical ceiling of the cheater, who sees the opponent's hand.

The AI against	**Games won (501 points)**	Hands won
Random	**83%**	63%
Heuristic	**72%**	59%
VoidExploit	**64%**	56%
TrumpCount	**62%**	56%

1,000 duplicate games per opponent, accurate to ±2 percentage points. It is the same AI, trained hand by hand: only the measurement changes between the two columns. In a single hand, the deal often decides; over a game, the better player wins much more often.

The learning curve

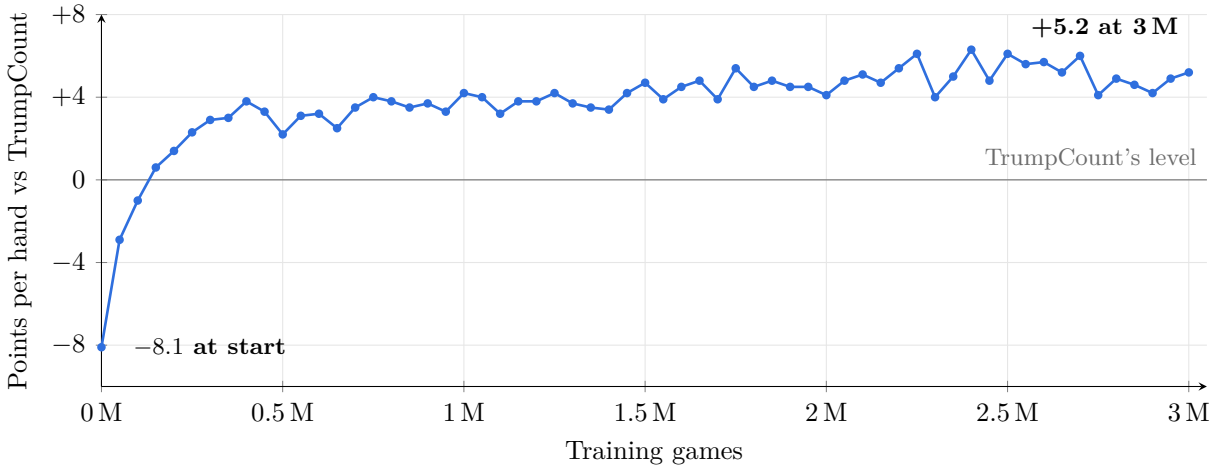


Figure 3: 3-million-game training run · 1,000 duplicate hands per point

The AI starts at −8 points per hand against TrumpCount, overtakes it after about 150,000 games (half an hour of training), then keeps improving more slowly.

The plateau

To check that the AI is really improving, and not just against our strategies, we made it play against its own versions saved during training.

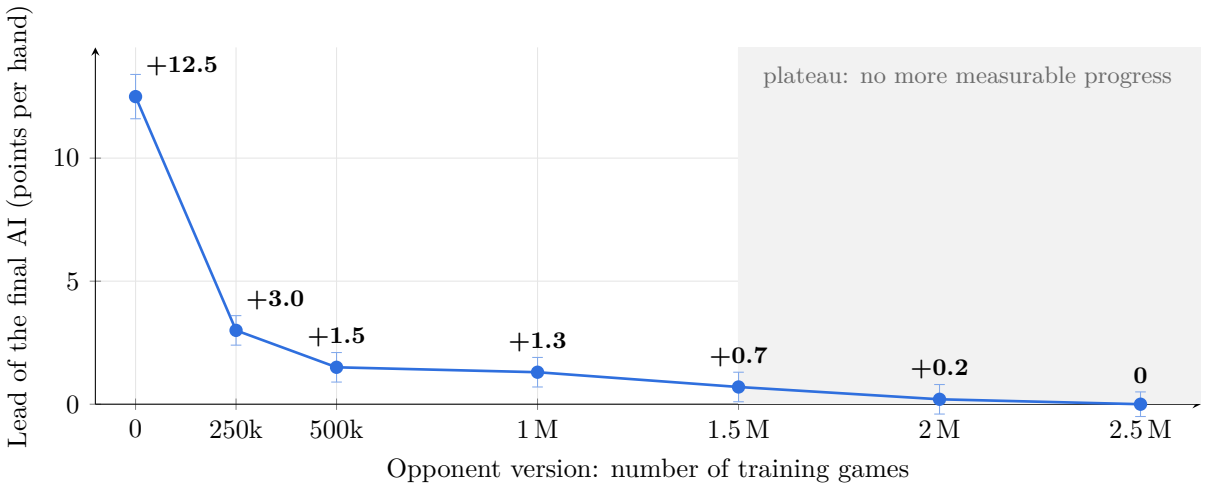


Figure 4: 1,000 duplicate deals per version

The final AI clearly beats its early versions, but no longer beats those from after 2 million games. Learning therefore plateaus around 1.5 to 2 million games: to go further, the method must change, not just the training length.

10. What the AI learned

By replaying deals move by move, we saw that the AI rediscovered on its own several reflexes of a good player:

- **waiting** for the opponent's big trumps to fall before playing its own;
- **trumping** when it pays off;
- **keeping its master cards** to cash them at the end of the hand.

An example

On one deal, trumps are diamonds. The player has strong trumps (Queen, 10, Ace and 8 of diamonds), but not the Jack or the 9, which the opponent holds.

Player	What it does	Points scored
TrumpCount	Leads the Ace of trumps, which is taken by the opponent's Jack (−31 points).	7
Our AI	Plays the 8 under the Jack, draws out the opponent's 9 with its Queen, then cashes its 10 and Ace, now masters.	**58**

A calculated bet, not a mistake

In 6.6% of its leads, the AI plays a 10 while the Ace of that suit has not yet fallen. On one lost deal, we first saw it as an oversight. Measuring over 3,000 games, we found that the opponent holds that Ace in only **15% of cases**: with 16 cards never dealt, the Ace is most often in nobody's hand. So it is a bet that pays off 85% of the time.

Lesson: never draw conclusions from a single deal, and measure a behaviour over thousands of games before calling it a mistake.

A test with no effect: the table of possible cards

To help the AI reason about the opponent's hand, we gave it a table of the cards the opponent may still hold, deduced from the rules (if it does not follow suit, it no longer has that suit, etc.). Over two new training runs, the result is the same: +5.1 points per hand against TrumpCount, versus +4.8 without the table, a gap smaller than the measurement margin.

The explanation: all this information could already be deduced from what the AI saw, and the network had learned those deductions by itself. This negative result is useful: it shows that the AI's limit lies elsewhere.

11. Trump bidding

The added rules

Instead of drawing trumps at random, both players choose them through a bid that is simpler than Coinche bidding, modelled on classic Belote:

1. One card is turned up from the 16 undealt cards and proposes its suit.
2. **First round**: each player takes that suit or passes.
3. **Second round**: each player names one of the other three suits or passes. If both pass again, the cards are redealt.
4. Whoever spoke first leads the first trick.
5. If the taker scores fewer points than the opponent, they are **"dedans"** (set): the opponent scores all the trick points.

The AI now chooses among 38 actions: the 32 cards, take, pass, and name one of the four suits. It learns to bid and to play at the same time. It starts from our best AI without bidding, so it does not relearn card play from scratch.

Results

Provisional results, after 350,000 training games out of the planned 2 million, in 501-point games with bidding on both sides:

Better bids than a hand-written rule. We built a baseline: our previous AI for card play, plus a bidding rule we wrote (take if the suit is strong enough in the hand).

Head to head, the AI that learned to bid wins **54.9% (±3.5)** of games against this baseline.

It takes more often, and is wrong less often. It takes trumps in 67% of hands, twice as often as TrumpCount (33%). Yet it is set less often when it takes: 13% of cases versus 19%.

The AI against	Games won
Random	98%
Heuristic	82%
VoidExploit	77%
TrumpCount	**73.5%**

Player	Games won against TrumpCount
Previous AI + hand-written bidding	68.8%
AI that learned to bid	**73.5%**

These figures cannot be compared directly with the 62% without bidding: with the "dedans" rule, every bad bid is very costly, which widens the gaps between players. Two 2-million-game training runs are under way to get the final figures.

12. Tools and app

The web app

An app lets you play against the AI in a browser, with real cards on screen. You click a card to play it, and only legal cards are clickable. A button reveals the AI's hand, and its confidence is shown at each move. The game is played to 501 points, with the score and a log of the tricks. The AI only sees what a human player would see.

Tests and reliability

- **9 automated test suites** check the game, each piece of PPO, trump bidding and the reliability of training runs. Every fixed bug has its own test, so it does not come back.
- **Resume after interruption**: an interrupted training run picks up exactly where it left off.
- **Safety stop**: if an invalid value appears, training stops and saves the faulty case for analysis. This tool came from a rare crash in the bidding training runs, which we reproduced and then fixed.
- **Overnight watchdog**: a program automatically restarts training runs that have stopped.